

ПОТОКИ

Владислав Богданов

2026-08-20

Оглавление

1. Введение	1
1.1. 1:1	1
1.2. N:M	1
1.3. N:1	1
2. POSIX	1
3. Советы ИИ на основе драйвера adt7140	3
3.1. Правило 1: Используй управляемые ресурсы (<code>devm_*</code>)	3
3.2. Правило 2: Храни состояние устройства в приватной структуре	3
3.3. Правило 3: Кэшируй чтение медленного железа (I2C/SPI)	4
3.4. Правило 4: Защищай доступ к общим данным мьютексом	5
3.5. Правило 5: Регистрируй атрибуты sysfs группами, а не по одному	5
3.6. Правило 6: Всегда проверяй возвращаемые значения на <code>IS_ERR</code>	6
3.7. Правило 7: Инкапсулируй низкоуровневый доступ к железу	6
3.8. Резюме	7
4. Создание задержек запуска потока	7

1. Введение

Windows

Приложение контейнер для потоков

Linux

Приложение = поток = процесс

Классификация потоков

критериев много но рассмотрим по **отображению в ядро**

1. 1:1
2. N:M
3. N:1

1.1. 1:1

Это поток созданный в любом процессе на любом уровне ОС управляется на прямую планировщиком ядра ОС. Он является полноценным потоком с точки зрения ОС. Именно эта технология используется в Linux.

1.2. N:M

Модель работает следующим образом, анализирует состояние системы и загрузку аппаратной части ПК и выполняет следующие действия, она часть пользовательских потоков выполняет по средствам библиотек, то есть в реальности с точки зрения самой системы все эти потоки являются одним процессом, а часть потоков если ресурсы системы позволяют на уровне планировщика ОС. Эта модель является гибридной моделью и довольно трудно реализуема так как требует требования к аппаратным ресурсам, примеры моделей обработка современной комп. графики, графические нейросети, тензорные сети и тд.

1.3. N:1

Все потоки пользовательские представляются одним процессом на уровне ядра. То есть у нас есть некая библиотека которая сама управляет всеми запущенными ресурсами, но при этом все эти ресурсы сами по себе расходуют аппаратные ресурсы машины. Самая медленная модель.

2. POSIX

В определенный момент в Linux появился инструментальный который соответствует стандарту POSIX. Это стандарт который в первую очередь определяет работу планировщика, на работы

системы, сигналов, прерываний и тд.

Как именно организовать работу систему?

Появились понятия:

1. lightweight-process или потоки

делят адресное пространство с породившим процессом, которые обладают связью с породившим процессом и обращаются к дескрипторам породившего их процесса и тд.

2. process

У него свое адресное пространство, свой набор дескрипторов

Это для пользовательского пространства, в ядре иначе.

В Linux всё работает по принципу строгой вложенности.

Вот кто кого формирует:

Потоки собираются в Процесс (в ядре это называется Группа потоков).

Процессы собираются в Группу процессов.

Группы процессов собираются в Сессию.

Как это происходит при запуске (кто кого порождает):



1. **Потоки в Процесс (Группа потоков):** Один главный поток плодит другие потоки (через *pthread_create* или *clone*). Вместе они образуют Группу потоков — то, что мы в обычной жизни называем "**одним процессом**". У них одна память.
2. **Процессы в Группе процессов:** Этот процесс может запустить дочерние процессы (через *fork*). Родитель и его дети образуют Группу процессов. Это нужно, например, чтобы разом убить всю цепочку команд по нажатию *Ctrl+C*.
3. **Группы процессов в Сессия:** Все эти процессы и группы, которые работают в одном окне терминала (или одной SSH-сессии), автоматически попадают в одну общую Сессию. Это нужно, чтобы разом закрыть их все, если ты закроешь окно терминала.

Короткая формула для запоминания:

Потоки в Процесс в Группе процессов в Сессия.

На уровне ядра любой поток является процессом.

3. Советы ИИ на основе драйвера adt7140

<https://elixir.bootlin.com/linux/v4.9.109/source/drivers/hwmon/adt7470.c#L1357>

Этот драйвер (`adt7470.c`) — настоящая сокровищница современных паттернов (шаблонов) разработки ядра Linux. Помимо идеальной работы с потоками, здесь реализованы лучшие практики управления памятью, синхронизации и взаимодействия с пользовательским пространством.

Вот 7 четких правил (шаблонов), которые можно извлечь из этого кода, с примерами:

3.1. Правило 1: Используй управляемые ресурсы (`devm_*`)

Суть: Никогда не используй обычные `kzalloc` / `kfree` или ручную регистрацию/отмену регистрации устройств, если это можно сделать через API `devm_` (device managed). Ядро автоматически освободит эти ресурсы при отключении устройства или ошибке в `probe`, что исключает утечки памяти и упрощает код.

Пример из кода:

```
// ПЛОХО (старый стиль):
data = kzalloc(sizeof(*data), GFP_KERNEL);
// ... много кода ...
kfree(data); // Нужно не забыть в remove и во всех ветках ошибок

// ХОРОШО (современный стиль из adt7470):
data = devm_kzalloc(dev, sizeof(struct adt7470_data), GFP_KERNEL);
if (!data)
    return -ENOMEM;

// Регистрация устройства тоже управляемая:
hwmon_dev = devm_hwmon_device_register_with_groups(dev, client->name, data,
adt7470_groups);
// При удалении драйвера ядро CAMO вызовет hwmon_device_unregister и kfree.
```

3.2. Правило 2: Храни состояние устройства в приватной структуре

Суть: Все данные, относящиеся к конкретному экземпляру устройства (кэши, мьютексы, указатели на клиент), должны быть упакованы в одну структуру `struct device_data`. Эта структура привязывается к объекту устройства через `dev_set_drvdata` / `i2c_set_clientdata`.

Пример из кода:

```
// 1. Объявление структуры контекста
struct adt7470_data {
    struct i2c_client *client;
    struct mutex lock;
    unsigned long sensors_last_updated;
    // ... другие поля ...
};

// 2. Привязка в probe
i2c_set_clientdata(client, data);

// 3. Получение в любом другом месте (например, в sysfs callback или потоке)
struct adt7470_data *data = i2c_get_clientdata(client);
// или
struct adt7470_data *data = dev_get_drvdata(dev);
```

3.3. Правило 3: Кэшируй чтение медленного железа (I2C/SPI)

Суть: Чтение регистров по медленным шинам (I2C) при каждом обращении из пользовательского пространства (sysfs) убивает производительность. Используй шаблон "валидности кэша" на основе `jiffies` и макроса `time_before`.

Пример из кода:

```
#define SENSOR_REFRESH_INTERVAL (5 * HZ) // Обновлять не чаще чем раз в 5 секунд

static struct adt7470_data *adt7470_update_device(struct device *dev)
{
    struct adt7470_data *data = dev_get_drvdata(dev);
    unsigned long local_jiffies = jiffies;
    int need_sensors = 1;

    // ШАБЛОН: Проверяем, прошло ли достаточно времени
    if (time_before(local_jiffies, data->sensors_last_updated +
        SENSOR_REFRESH_INTERVAL) &&
        data->sensors_valid) {
        need_sensors = 0; // Данные свежие, читать с железа не нужно!
    }

    if (need_sensors) {
        // ... медленное чтение с I2C ...
        data->sensors_last_updated = local_jiffies; // Обновляем метку времени
        data->sensors_valid = 1;
    }
}
```

```
}  
    return data;  
}
```

3.4. Правило 4: Защищай доступ к общим данным мьютексом

Суть: Если данные устройства могут быть изменены из разных контекстов (например, фоновый поток обновляет кэш, а пользовательский процесс читает его через sysfs), доступ к этим данным **всегда** должен быть защищен **mutex**.

Пример из кода:

```
// Инициализация в probe:  
mutex_init(&data->lock);  
  
// Использование при чтении/записи:  
static ssize_t set_temp_max(struct device *dev, ...) {  
    // ...  
    mutex_lock(&data->lock);          // 1. Захватываем  
    data->temp_max[attr->index] = temp;  
    i2c_smbus_write_byte_data(...);  // 2. Работаем с железом и памятью  
    mutex_unlock(&data->lock);        // 3. Освобождаем  
    // ...  
}
```

Примечание: В потоке обновления (**adt7470_update_thread**) мьютекс также захватывается на время чтения с I2C, чтобы sysfs не прочитал "наполовину" обновленные данные.

3.5. Правило 5: Регистрируй атрибуты sysfs группами, а не по одному

Суть: Старый стиль (**device_create_file**) требует вручную удалять каждый файл в **remove**. Современный стандарт — использовать макросы **DEVICE_ATTR** / **SENSOR_DEVICE_ATTR**, объединять их в массив и регистрировать одной функцией ***_register_with_groups**.

Пример из кода:

```
// 1. Объявление атрибутов  
static SENSOR_DEVICE_ATTR(temp1_max, S_IWUSR | S_IRUGO, show_temp_max, set_temp_max,  
0);  
static SENSOR_DEVICE_ATTR(temp2_max, S_IWUSR | S_IRUGO, show_temp_max, set_temp_max,  
1);
```

```
// 2. Группировка в массив (заканчивается NULL)
static struct attribute *adt7470_attrs[] = {
    &sensor_dev_attr_temp1_max.dev_attr.attr,
    &sensor_dev_attr_temp2_max.dev_attr.attr,
    // ...
    NULL
};

// 3. Создание группы
ATTRIBUTE_GROUPS(adt7470); // Этот макрос создает adt7470_groups

// 4. Регистрация одной строкой (и автоматическая очистка благодаря devm_)
hwmon_dev = devm_hwmon_device_register_with_groups(dev, client->name, data,
adt7470_groups);
```

3.6. Правило 6: Всегда проверяй возвращаемые значения на **IS_ERR**

Суть: Функции ядра, которые возвращают указатель (например, `kthread_run`, `devm_kzalloc`, `hwmon_device_register`), при ошибке возвращают не `NULL`, а закодированный указатель на ошибку (например, `-ENOMEM`). Проверяй их макросом `IS_ERR`, а извлекай код ошибки макросом `PTR_ERR`.

Пример из кода:

```
data->auto_update = kthread_run(adt7470_update_thread, client, "%s",
dev_name(hwmon_dev));
if (IS_ERR(data->auto_update)) {
    return PTR_ERR(data->auto_update); // Корректно вернет, например, -ENOMEM
}
```

3.7. Правило 7: Инкапсулируй низкоуровневый доступ к железу

Суть: Не размазывай логику чтения/записи регистров (особенно если есть нюансы, вроде порядка байт) по всему коду. Выноси это в `static inline` функции-хелперы.

Пример из кода:

```
/* 16-битные регистры читаются младшим байтом первым.
   Выносим эту логику в одно место */
static inline int adt7470_read_word_data(struct i2c_client *client, u8 reg)
```



```
{
    u16 foo;
    foo = i2c_smbus_read_byte_data(client, reg);
    foo |= ((u16)i2c_smbus_read_byte_data(client, reg + 1) << 8);
    return foo;
}
```

```
// Использование в коде становится чистым и понятным:
data->fan[i] = adt7470_read_word_data(client, ADT7470_REG_FAN(i));
```

3.8. Резюме

Если вы будете писать драйвер, следуя этим 7 правилам + правильному шаблону **kthread** (который мы разобрали ранее), ваш код будет: 1. **Безопасным** (нет утечек памяти благодаря **devm_**, нет гонок данных благодаря **mutex**). 2. **Производительным** (благодаря кэшированию через **time_before**). 3. **Читаемым** (благодаря группам атрибутов и хелперам). 4. **Готовым к мейнтейнсу** (мейнтейнеры ядра любят именно такой стиль и с большей вероятностью примут ваш патч).

4. Создание задержек запуска потока

Методы:

1. **spin-lock** - самый простой и самый редко применяемый метода, блокировка в цикле ожидания, крутимся внутри **while**

```
while(условие)
    schedule(); //помещает в очередь планировщика
```

но есть **spin_lock** и **spin_unlock()** - применяется в ядре для обработки прерываний но смысл такой же.

Для условия используем мьютекс.